Centralized BI Portal

Governed, AI-native dashboards for every business function

Your name · AI Data Platform & Engineering · date

Okta OAuth2 + PKCE · Centralized RBAC · Claude-powered insights · Single-container deploy

The Problem

- Every business function (Finance, Sales Ops, RevOps) needs dashboards — but can't build them
- The data team becomes the **permanent bottleneck**: every ask is a ticket, every ticket is a delay
- The unsafe workaround: one-off exports, emailed spreadsheets — **no governance, no trust, no audit trail**
- The challenge: give teams **self-serve access** without losing the access controls and review gates the data platform team needs

What We Built (scope on one slide)

Five pillars, each fully implemented:

1. **Portal shell** — Shared, Domain (per business function), and Personal workspaces
2. **Auth** — Okta OAuth2 Authorization Code + PKCE, with a zero-dependency mock fallback
3. **RBAC** — centralized, server-side, tested in isolation; enforced at every API layer
4. **Governed publishing** — Draft → In Review → Published state machine with explicit audit records
5. **Two Claude dashboards** — ARR Waterfall (Finance) + Pipeline Health (Sales Ops); chart + grounded AI insight
6. **Deployable** — single Docker image → Kubernetes or Snowflake Container Services

Architecture

- **One container** — FastAPI serves both the API and the built React SPA. Same origin in production: no CORS, no token forwarding.
- **Pluggable externals** — Okta, Snowflake, Anthropic/Claude, Postgres/SQLite — each has a graceful fallback with identical code paths.
- **Stateless app tier** — signed-cookie sessions (itsdangerous). No shared session store needed → horizontal scaling.

```
flowchart LR
  subgraph Browser
    SPA[React SPA<br/>Recharts + Tailwind]
  end
  subgraph Container["BI Portal container"]
    direction TB
    API[FastAPI]
    AUTH[Auth: OAuth2+PKCE<br/>signed sessions]
    RBAC[RBAC core<br/>Principal + authz]
    DASH[Dashboard service]
    SNOW[Snowflake service]
    CLAUDE[Claude service]
    ORM[(SQLAlchemy<br/>governance DB)]
    API --> AUTH --> RBAC
    API --> DASH --> SNOW
    DASH --> CLAUDE
    API --> ORM
  end
  Okta[(Okta IdP)]
  SF[(Snowflake ANALYTICS)]
  ANT[(Anthropic Claude API)]
  PG[(Postgres / SQLite)]
  SPA -->|"/api, cookie"| API
  AUTH -->|OIDC discovery, token, JWKS| Okta
  SNOW -->|parameterized SQL| SF
  CLAUDE -->|messages API| ANT
  ORM --> PG
```

Live Demo (~4 min)

1. Log in as **Frank (Finance analyst)** → sees Finance + Shared folders; Sales Ops is absent
2. Open **ARR Waterfall** → chart loads, Claude insight panel appears alongside it
3. Try to open a **Sales Ops dashboard by direct URL** → **404 —** not a hidden button, the API genuinely doesn't return the data
4. As Frank, create a draft in **Personal Workspace** → submit for review
5. Switch to **Fiona (Finance Approver)** → **Approvals** tab → approve → report moves to the Finance domain folder

> *The 404 on the direct URL is the moment to dwell on — that's not a frontend trick, that's the API enforcing access before touching the database. I'll show the test that proves it.*

Security: Okta OAuth2 + PKCE

- **Authorization Code + PKCE** (OAuth 2.1) — no implicit flow, no client secret in the browser
- `state` parameter: CSRF protection on the redirect
- `nonce` claim: replay protection — binds the `id_token` to this specific login attempt
- JWKS signature verification: `iss`, `aud`, `nonce`, `RS256` — we verify, not just decode
- Group memberships arrive in the `groups` claim of the `id_token` → only input to RBAC
- Mock fallback mints the exact same session structure — same downstream code path

```
sequenceDiagram
  participant U as Browser
  participant P as Portal (FastAPI)
  participant O as Okta
  U->>P: GET /api/auth/login
  P->>P: generate PKCE (verifier, challenge), state, nonce
  P-->>U: 302 to Okta /authorize?code_challenge=...&state=...
  Note over P,U: verifier/state/nonce stored in short-lived signed cookie
  U->>O: authenticate + consent
  O-->>U: 302 /api/auth/callback?code=...&state=...
  U->>P: GET /api/auth/callback
  P->>P: verify state (CSRF)
  P->>O: POST /token (code + code_verifier)
  O-->>P: id_token (JWT, contains groups claim)
  P->>O: fetch JWKS, verify signature/iss/aud/nonce
  P->>P: claims -> Principal (groups -> domains/roles)
  P-->>U: Set signed session cookie, 302 to /
```

RBAC: How It Actually Works

- Group string → `(domain, role)` mapping lives in **exactly one file** (`auth/rbac.py`). No router ever parses a group string.
- **Two enforcement layers** — defense in depth: point checks + query scoping
- Cross-domain access returns **404, not 403** — the API does not confirm the resource exists

Okta group	Domain	Role
`BI-Portal-Admin`	all	admin
`BI-Finance`	finance	author
`BI-Finance-Approver`	finance	approver
`BI-SalesOps`	sales_ops	author
`BI-SalesOps-Approver`	sales_ops	approver
`BI-RevOps`	revops	author
`BI-CustomerSuccess`	customer_success	author

```
HTTP request
  → get_current_principal()       # 401 if no/invalid session
  → get_current_user()            # resolve DB user (JIT provisioned)
  → authz.* check on the resource # 404/403 if not entitled
  → query scoping (visible_*)     # list endpoints can't over-return
  → handler runs
  → audit() security-relevant actions
```

Governed Publishing Workflow

- Reports are born in a **Personal Workspace** (private to the author)
- Author submits → status moves to **In Review**; a domain approver reviews
- Approver approves → report moves to the **Domain folder** (visible to all domain members)
- Rejects → **Changes Requested** (back to the author)
- Every submission and decision is an **explicit** `ApprovalRequest` **row** — not a boolean flag

```
stateDiagram-v2
  [*] --> DRAFT: create in Personal Workspace
  DRAFT --> IN_REVIEW: author submits (target domain)
  CHANGES_REQUESTED --> IN_REVIEW: author resubmits
  IN_REVIEW --> PUBLISHED: approver approves → moved to Domain Workspace
  IN_REVIEW --> CHANGES_REQUESTED: approver requests changes
  PUBLISHED --> [*]
```

The Claude Dashboards: Engineering the AI Layer

Dashboard	Stakeholder	Mental model
ARR Waterfall	Finance / FP&A	Beginning ARR → bridge components → ending ARR. NRR, GRR, churn.
Pipeline Health	Sales Ops	Funnel by stage, deal aging buckets, slippage, coverage ratio vs quota.

Each dashboard: **structured data in → Recharts chart + Claude insight panel out**

The insight panel returns: `headline` · `narrative` · `key_findings` · `risks` · `recommended_actions`

AI Prompt Engineering: Making It Trustworthy

System prompt pins: persona (senior analyst), audience (VP/exec), grounding rule (*"use ONLY the numbers provided"*), output contract (strict JSON)

User prompt passes: the **exact same JSON the chart renders from** (no transcription gap)

Defensive decode: strips stray fences, locates JSON, validates required keys

Failure	Response
No API key	Deterministic mock insight (reads real metrics, labeled `mock:no_api_key`)
API error / timeout	Exponential backoff (3 attempts), then mock
Malformed JSON	Defensive parse → repair-or-mock
Model invents a number	System-prompt grounding rule + verbatim data in; outputs cite specific figures so hallucinations are visible

Data Model

```
erDiagram
  USER ||--o{ FOLDER : "owns (personal)"
  USER ||--o{ REPORT : authors
  FOLDER ||--o{ REPORT : contains
  REPORT ||--o{ APPROVAL_REQUEST : has
  USER { int id PK; string okta_sub UK; string email; string name }
  FOLDER { int id PK; string slug UK; enum type "shared|domain|personal"; string domain; int owner_user_id FK }
  REPORT { int id PK; enum dashboard_type; enum status; int folder_id FK; int owner_user_id FK; string target_domain; json config }
  APPROVAL_REQUEST { int id PK; int report_id FK; string target_domain; enum decision; int reviewer_id }
  AUDIT_EVENT { int id PK; string user_email; string action; bool allowed }
```

Key design: `target_domain` (intent, fixed at creation) is **separate** from `folder_id` (location, moves on publish). Publishing = an auditable folder move, not a status toggle.

Analytical data **stays in Snowflake** — this store holds only governance metadata.

Tests: Proving the Security Model

`test_rbac.py` — unit tests (no HTTP, no DB):

- Admin sees all domains; domain member is author-only; `-Approver` suffix elevates role; no groups → no access

`test_api_rbac.py` — end-to-end through `TestClient`:

- Unauthenticated → 401 on every protected endpoint
- Finance user sees Finance + Shared, **not** Sales Ops
- Finance user hitting Sales Ops by direct URL → **404** (folder list and dashboard render)
- Non-approver sees empty approval queue; approver sees the pending item
- Dashboard renders: bridge exists, insight has headline, label is `mock` or `claude`

> *These are the **proofs**, not the claims. The unit tests run in < 1s with no network — the e2e tests run the full stack in memory.*

Deployment

One image, two targets:

- **Kubernetes** (`deploy/k8s/`) — ConfigMap (non-secrets), Secret via External Secrets, `replicas: 2` , readiness + liveness on `/api/health` , non-root UID, `readOnlyRootFilesystem` , dropped capabilities, TLS Ingress
- **Snowflake Container Services** (`deploy/spcs/`) — compute pool, least-privilege `BI_PORTAL_READER` role, native `SECRET` objects, external-access integration for Okta + Anthropic, `CREATE SERVICE`

Secrets **never in the image** — rotating a secret = a rollout, not a rebuild.

```
flowchart TB
  subgraph K8s["Kubernetes (or SPCS)"]
    ING[Ingress + TLS] --> SVC[Service]
    SVC --> P1[Pod: bi-portal]
    SVC --> P2[Pod: bi-portal]
    CM[ConfigMap<br/>non-secret] -.envFrom.-> P1
    SEC[Secret / External Secrets<br/>Okta, Claude, DB] -.envFrom.-> P1
  end
  P1 --> OKTA[(Okta)]
  P1 --> ANT[(Anthropic)]
  P1 --> SF[(Snowflake)]
  P1 --> DB[(Postgres)]
```

What I'd Build Next

Highest leverage: conversational "ask-your-data" over a governed semantic layer

- Stakeholder types *"What drove SMB churn last quarter?"* → chart + answer
- Claude does **NL → constrained query object** against a YAML/dbt-metrics semantic layer (not free SQL)
- The query inherits the caller's RBAC + Snowflake row-access policies — you cannot ask your way past a permission boundary

Then:

- Scheduled refresh + pre-generated insights (remove Claude latency from the request path)
- dbt lineage + source freshness trust signals on every dashboard
- Audit console (the `AuditEvent` table is already there — surface it)
- Redis-backed sessions for server-side revocation; SCIM deprovisioning
- Self-serve dashboard template registry — new types as config + a small module

Appendix — Keep Ready for Q&A

Slide	Topic
A1	Why FastAPI / signed-cookie sessions / SPA-served-by-API — trade-offs table
A2	Walk `test_api_rbac.py::test_cross_domain_direct_url_is_404` line by line
A3	Snowflake reference SQL + row-access-policy design
A4	Claude cost controls: cache by `(dashboard_type, data_hash, model)`, async pre-generation, model tiering
A5	Migration path: SQLite → Postgres + Alembic; signed cookies → Redis session IDs
A6	Prompt evaluation harness: labelled `(data → expected_finding)` set run in CI
A7	Why 404 not 403 for cross-domain access — enumeration attack surface

Presentation tips:

- Lead with the problem + demo (slides 2, 5) before architecture
- The cross-domain 404 (slide 5) is the most convincing moment — say *"the test for this is slide 12"*
- *Head of Eng:* slides 6, 7, 12, 13 · *AI Engineers:* 9, 10 · *BI Leads:* 2, 5, 8, 9 · *Data Platform:* 4, 7, 11, 13